

Team Notebook

[UNI-FIIS] - SaleEnOdeUno

1 Math	2
1.1 ntt	2
1.2 fwht bitwiseconvolution	2
1.3 diophantine	3
1.4 criba	3
1.5 geo2d	3
1.6 xor basis	4
1.7 fft	4
1.8 millerrabin polardrho	5
1.9 primitive root	5
2 Strings	5
2.1 prefix function	5
2.2 hashing	6
2.3 z function	6
3 Graphs	6
3.1 tree bridge	6
3.2 2sat	6
3.3 dinic	6
4 Data Structure	7
4.1 2d fenwick tree	7
4.2 dynamic segment tree	7
4.3 pst	7
4.4 segment tree 2d	8
4.5 merge sort tree	8
4.6 ordered set	9
4.7 sparse table	9
4.8 pst lazy	9
4.9 fenwick tree	9
4.10 centroid decomposition	9
4.11 multi ordered set	10
4.12 hld	10

4.13 segment tree	10
4.14 hld edge	10

Math

1.1 ntt

```
const ll MOD = 998244353;
vector<vector<int>> w;
int pow2, root;
int smod(int a, int b, int mod) {return (a + b >= mod ? a + b - mod : a + b);}
int rmod(int a, int b, int mod) {return (a - b < 0 ? a - b + mod : a - b);}
int mult(int a, int b, int mod) {return (ll)a * b % mod;}
int qp(int a, int e, int mod) {
    int ans = 1;
    while (e) {
        if (e & 1) ans = mult(ans, a, mod);
        a = mult(a, a, mod);
        e >>= 1;
    }
    return ans;
}
void calc(int log, int mod) {
    w.resize(log + 1);
    w[0].resize(1, 1);
    for (int l = 1, len = 2; l <= log; l++, len <= 1) {
        w[l].resize(len / 2);
        int wn = qp(root, pow2 / len, mod);
        for (int j = 0; 2 * j < len; j++) {
            if (j & 1) w[l][j] = mult(w[l - 1][j / 2], wn, mod);
            else w[l][j] = w[l - 1][j / 2];
        }
    }
}
void findroot(int mod) {
    pow2 = 1;
    int k = mod - 1;
    while (!(k & 1)) pow2 <= 1, k >>= 1;
    root = 1;
    while (qp(root, pow2, mod) != 1 || qp(root, pow2 / 2, mod) == 1) root++;
}
void ntt(vector<int> &a, bool invert, int mod) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int l = 1, len = 2; len <= n; l++, len <= 1)
        for (int i = 0; i < n; i += len)
            for (int j = 0; 2 * j < len; j++) {
                int u = a[i + j], v = mult(a[i + j + len /
```

```
2], w[l][j], mod);
                a[i + j] = smod(u, v, mod);
                a[i + j + len / 2] = rmod(u, v, mod);
            }
        }
        if (invert) {
            reverse(a.begin() + 1, a.end());
            int invn = qp(n, mod - 2, mod);
            for (int i = 0; i < n; i++) a[i] = mult(a[i], invn, mod);
        }
    }
    vector<int> polymultmod(vector<int> a, vector<int> b, int mod = MOD) {
        int n = 1, log = 0, rn = a.size() + b.size() - 1;
        while (n < a.size() + b.size() - 1) n <= 1, log++;
        findroot(mod);
        calc(log, mod);
        a.resize(n), b.resize(n);
        ntt(a, 0, mod), ntt(b, 0, mod);
        for (int i = 0; i < n; i++) a[i] = mult(a[i], b[i], mod);
        ntt(a, 1, mod);
        a.resize(rn);
        return a;
    }
}
```

1.2 fwht bitwiseconvolution

```
//OR-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <= 1, h <= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) a[j + h] += a[j];
}
void ifwht(vector<ll> &a) {
    int n = A.size();
    for (int s = n, h = s >> 1; s >= 2; s >>= 1, h >>= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) A[j + h] -= A[j];
}
vector<ll> OrConvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
vector<ll> SubsetConvolution(vector<ll> a, vector<ll> b) {
    int n = 1, bit = 1;
```

```
while (n < a.size() || n < b.size()) n <= 1, bit++;
a.resize(n), b.resize(n);
vector<vector<ll>> A(bit + 1, vector<ll>(n)), B(bit + 1, vector<ll>(n));
for (int i = 0; i < n; i++) {
    int f = __builtin_popcount(i);
    A[f][i] = a[i], B[f][i] = b[i];
}
for (int i = 0; i <= bit; i++) fwht(A[i]), fwht(B[i]);
vector<ll> conv(n, 0);
for (int k = 0; k < n; k++) {
    int f = __builtin_popcount(k);
    for (int i = 0; i <= f; i++) conv[k] += A[i][k] * B[f - i][k];
}
ifwht(conv);
return conv;
}
//AND-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <= 1, h <= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) a[j] += a[j + h];
}
void ifwht(vector<ll> &a) {
    int n = A.size();
    for (int s = n, h = n >> 1; s >= 2; s >>= 1, h >>= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) A[j] -= a[j + h];
}
vector<ll> Andconvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
//XOR-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <= 1, h <= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) {
                ll a1 = a[j + h], a0 = a[j];
                a[j + h] = a0 - a1, a[j] = a0 + a1;
            }
}
void ifwht(vector<ll> &a) {
    fwht(A);
```

```

int n = A.size();
for (int i = 0; i < n; i++) A[i] /= n;
}
vector<ll> Xorconvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}

```

1.3 diophantine

```

ll extended_gcd(ll a, ll b, ll &x, ll &y) {
    x = 1, y = 0;
    ll x1 = 0, y1 = 1;
    while (b) {
        ll q = a / b;
        tie(x, x1) = make_pair(x1, x - q * x1);
        tie(y, y1) = make_pair(y1, y - q * y1);
        tie(a, b) = make_pair(b, a - q * b);
    }
    return a < 0 ? (x *= -1, y *= -1, a *= -1, a) : a;
}
bool diophantine(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
    if (a == 0 && b == 0) return c ? 0 : (x = y = 0, 1);
    g = extended_gcd(a, b, x, y);
    return (c % g == 0 ? (x *= c / g, y *= c / g, 1) : 0);
}
ll divdown(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b);
}
ll divup(ll a, ll b) {
    return a / b + ((a ^ b) >= 0 && a % b);
}
vector<pll> allsolution(ll a, ll b, ll c, ll lx, ll rx, ll ly, ll ry) {
    // x = x0 + k*(b/g)    y = y0 - k*(a/g)
    ll x0, y0, g;
    if (!diophantine(a, b, c, x0, y0, g)) return {};
    vector<pll> ans;
    if (!g) {
        for (ll x = lx; x <= rx; x++)
            for (ll y = ly; y <= ry; y++) ans.pb({x, y});
        return ans;
    }
    if (!a) {
        ll y = c / b;
        if (!(ly <= y && y <= ry)) return {};
        for (ll x = lx; x <= rx; x++) ans.pb({x, y});
        return ans;
    }
}

```

```

if (!b) {
    ll x = c / a;
    if (!(lx <= x && x <= rx)) return {};
    for (ll y = ly; y <= ry; y++) ans.pb({x, y});
    return ans;
}
ll dx = b / g, dy = a / g;
ll kmin, kmax;
if (dx > 0) kmin = divup(lx - x0, dx), kmax = divdown(rx - x0, dx);
else kmin = divup(rx - x0, dx), kmax = divdown(lx - x0, dx);
if (dy > 0) kmin = max(kmin, divup(y0 - ry, dy)), kmax = min(kmax, divdown(y0 - ly, dy));
else kmin = max(kmin, divup(y0 - ly, dy)), kmax = min(kmax, divdown(y0 - ry, dy));
for (ll k = kmin; k <= kmax; k++) ans.pb({x0 + k * dx, y0 - k * dy});
return ans;
}
bool positive_sol_minf(ll a, ll b, ll c, ll &x, ll &y, ll cx, ll cy) {
    // f(x,y)= cx*x + cy*y
    ll g;
    if (!diophantine(a, b, c, x, y, g)) return 0;
    ll dx = b / g, dy = a / g;
    ll kmin = -INF, kmax = INF;
    dx > 0 ? kmin = max(kmin, divup(-x, dx)) : kmax = min(kmax, divdown(-x, dx));
    dy < 0 ? kmin = max(kmin, divup(y, dy)) : kmax = min(kmax, divdown(y, dy));
    if (kmin > kmax) return 0;
    ll df = cx * dx - cy * dy;
    ll k = df >= 0 ? kmin : kmax; // df<0 para max
    x += dx * k;
    y -= dy * k;
    return 1;
}
}

```

1.4 criba

```

bool criba[MX];
ll primerfactor[MX], phi[MX];
// criba para primerfactor, phi 0(nlog(log(n)))
void completar() {
    for (int i = 0; i < MX; i++) {
        criba[i] = 1;
        primerfactor[i] = i;
        phi[i] = i;
    }
    criba[0] = 0;
    criba[1] = 0;
    for (ll i = 2; i < MX; i++) {
        if (criba[i]) {
            phi[i]--;

```

```

        for (ll j = 2 * i; j < MX; j += i) {
            criba[j] = 0;
            phi[j] -= phi[j] / i;
            if (primerfactor[j] == j) primerfactor[j] = i;
        }
    }
}
vector<pll> factorizacioncriba(ll n) { // 0(log(n))
    vector<pll> ans;
    while (n > 1) {
        ll f = primerfactor[n];
        ll pot = 0;
        while (primerfactor[n] == f) {
            n /= f;
            pot++;
        }
        ans.pb({f, pot});
    }
    return ans;
}
vector<ll> divisorescriba(ll n) { // 0(d(x))
    vector<ll> fact = {1};
    while (n > 1) {
        ll f = primerfactor[n];
        ll sz = fact.size();
        ll divisor = 1;
        while (n % f == 0) {
            divisor *= f;
            n /= f;
            rep(i, sz) fact.pb(fact[i] * divisor);
        }
        return fact;
    }
}

```

1.5 geo2d

```

struct point {
    ll x, y;
    point() {}
    point(ll _x, ll _y) : x(_x), y(_y) {}
    point operator+(const point &p) const {
        return {x + p.x, y + p.y};
    }
    point operator-(const point &p) const {
        return {x - p.x, y - p.y};
    }
    bool operator==(const point &p) const {
        return x == p.x && y == p.y;
    }
    bool operator<(const point &p) const {
        if (x == p.x) return y < p.y;
        return x < p.x;
    }
}

```

```

    }
};
// Producto punto
long long dot(const point &a, const point &b) {
    return a.x * b.x + a.y * b.y;
}
// Producto cruzado (determinante)
long long cross(const point &a, const point &b) {
    return a.x * b.y - a.y * b.x;
}
// Cross entre vectores AB y AC
long long cross(const point &a, const point &b, const point &c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}
// El punto p está en el segmento ab?
bool onSegment(const point &a, const point &b, const point &p) {
    if (cross(a, b, p) != 0) return false;
    return min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
        min(a.y, b.y) <= p.y &&
        p.y <= max(a.y, b.y);
}
// Interseccion de 2 segmentos
bool intersect(const point &p1, const point &q1, const point &p2, const point &q2) {
    long long d1 = cross(p1, q1, p2);
    long long d2 = cross(p1, q1, q2);
    long long d3 = cross(p2, q2, p1);
    long long d4 = cross(p2, q2, q1);
    if ((d1 > 0 && d2 < 0 || d1 < 0 && d2 > 0) && (d3 > 0 && d4 < 0 || d3 < 0 && d4 > 0))
        return true;
    if (d1 == 0 && onSegment(p1, q1, p2)) return true;
    if (d2 == 0 && onSegment(p1, q1, q2)) return true;
    if (d3 == 0 && onSegment(p2, q2, p1)) return true;
    if (d4 == 0 && onSegment(p2, q2, q1)) return true;
    return false;
}
bool pointInPolygon(const vector<point> &poly, const point &p) {
    bool inside = false;
    int n = poly.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        const point &a = poly[i];
        const point &b = poly[j];
        if (onSegment(a, b, p)) return true;
        if ((a.y > p.y) != (b.y > p.y)) {
            long long x_intersection = (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x);
            long long y_diff = b.y - a.y;
            bool right = (x_intersection * y_diff) < 0;
            if (right) inside = !inside;
        }
    }
}

```

```

    }
    return inside;
}

```

1.6 xor basis

```

template <typename T, int LOG>
struct Basis {
    T basis[LOG] = {0};
    int dim = 0;
    bool insert(T x) { // Formará parte del Basis?
        for (int i = LOG - 1; i >= 0; i--)
            if ((x >> i) & 1) {
                if (basis[i] x ^= basis[i];
                else {
                    basis[i] = x, dim++;
                    return true;
                }
            }
        return false;
    }
    T size() {
        return ((T)1 << dim);
    }
    bool find(T x) { // ¿x ∈ span{x1,x2...}?
        for (int i = LOG - 1; i >= 0; i--) x = min(x, x ^ basis[i]);
        return x == 0;
    }
    T max_xor(T x = 0) { // max (x^a / a ∈ span{x1,x2...})
        for (int i = LOG - 1; i >= 0; i--) x = max(x, x ^ basis[i]);
        return x;
    }
    T kth(T k) { // The k_smaller of span{x1,x2...}
        if (k < 1 || k > ((T)1 << dim)) return -1;
        T x = 0, cont = ((T)1 << dim);
        for (int i = LOG - 1; i >= 0; i--)
            if (basis[i]) {
                if (k > (cont >> 1)) {
                    if (!(x >> i) & 1) x ^= basis[i];
                    k -= (cont >> 1);
                } else if ((x >> i) & 1) x ^= basis[i];
                cont >>= 1;
            }
        return x;
    }
}

```

1.7 fft

```

typedef complex<double> cd;
vector<vector<cd>> w;
void calc(int log) {
    w.resize(log + 1);
}

```

```

w[0].resize(1, 1);
for (int l = 1, len = 2; l <= log; l++, len <= len * 2) {
    w[l].resize(len / 2);
    cd wn(cos(2 * PI / len), sin(2 * PI / len));
    for (int j = 0; 2 * j < len; j++)
        w[l][j] = (j & 1 ? w[l - 1][j / 2] * wn : w[l - 1][j / 2]);
}
}
void fft(vector<cd> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int l = 1, len = 2; len <= n; l++, len <= len * 2)
        for (int i = 0; i < n; i += len)
            for (int j = 0; 2 * j < len; j++) {
                cd u = a[i + j], v = a[i + j + len / 2] *
                    (j & 1 ? 1 : -1);
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
            }
    if (invert) {
        reverse(a.begin() + 1, a.end());
        for (int i = 0; i < n; i++) a[i] /= n;
    }
}
vector<ll> polymult(vector<ll> &a, vector<ll> &b) {
    int n = 1, log = 0;
    while (n < a.size() + b.size() - 1) n <= n * 2, log++;
    calc(log);
    vector<cd> A(n);
    for (int i = 0; i < n; i++) A[i] = cd(i < a.size() ? a[i] : 0, i < b.size() ? b[i] : 0);
    fft(A, 0);
    for (int i = 0; i < n; i++) A[i] *= A[i];
    fft(A, 1);
    vector<ll> ans(a.size() + b.size() - 1);
    for (int i = 0; i < ans.size(); i++) ans[i] = round(A[i].imag() / 2);
    return ans;
}
// 2D-FFT
void fft2D(vector<vector<cd>> &a, bool invert) {
    int n = a.size(), m = a[0].size();
    for (int i = 0; i < n; i++) fft(a[i], invert);
    vector<cd> b(n);
    for (int j = 0; j < m; j++) {
        for (int i = 0; i < n; i++) b[i] = a[i][j];
        fft(b, invert);
        for (int i = 0; i < n; i++) a[i][j] = b[i];
    }
}

```

```

}
vector<vector<ll>> convolution2D(vector<vector<ll>> &a,
vector<vector<ll>> &b) {
    int n = 1, logn = 0;
    while (n < a.size() + b.size() - 1) n <= 1, logn++;
    int m = 1, logm = 0;
    while (m < a[0].size() + b[0].size() - 1) m <= 1, logm++;
    calc(max(logn, logm));
    vector<vector<cd>> A(n, vector<cd>(m));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            A[i][j] = cd(i < a.size() && j < a[0].size() ?
a[i][j] : 0,
i < b.size() && j < b[0].size() ?
b[i][j] : 0);
    fft2D(A, 0);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) A[i][j] *= A[i][j];
    fft2D(A, 1);
    vector<vector<ll>> ans(a.size() + b.size() - 1,
vector<ll>(a[0].size() + b[0].size() - 1));
    for (int i = 0; i < ans.size(); i++)
        for (int j = 0; j < ans[0].size(); j++) ans[i][j] =
round(A[i][j].imag() / 2);
    return ans;
}

```

1.8 millerrabin polardrho

```

ll mult(ll a, ll b, ll mod) {
    return (int128)a * b % mod;
}
ll qp(ll a, ll e, ll mod) {
    ll ans = 1;
    while (e) {
        if (e & 1) ans = mult(ans, a, mod);
        a = mult(a, a, mod);
        e >>= 1;
    }
    return ans;
}
bool compositetest(ll a, ll d, ll s, ll n) {
    ll x = qp(a, d, n);
    if (x == 1 || x == n - 1) return 0;
    for (ll i = 0; i < s - 1; i++) {
        x = mult(x, x, n);
        if (x == n - 1) return 0;
    }
    return 1;
}
bool esprimo(ll n) {
    if (n < 2) return 0;
    ll d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, s++;
}

```

```

for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
37}) {
    if (n == a) return 1;
    if (compositetest(a, d, s, n)) return 0;
}
return 1;
}
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
ll random(ll a, ll b) {
    return uniform_int_distribution<ll>(a, b)(rng);
}
ll f(ll x, ll c, ll mod) {
    ll m = mult(x, x, mod);
    return m + c >= mod ? m + c - mod : m + c;
}
ll getfactor(ll n) {
    if (n % 2 == 0) return 2;
    if (esprimo(n)) return n;
    while (true) {
        ll y = random(1, n - 1);
        ll c = random(1, n - 1);
        ll m = 64, g = 1, r = 1, q = 1;
        ll x, ys;
        while (g == 1) {
            x = y;
            for (ll i = 0; i < r; i++) y = f(y, c, n);
            ll k = 0;
            while (k < r && g == 1) {
                ys = y;
                ll lim = min(m, r - k);
                for (ll i = 0; i < lim; i++) {
                    y = f(y, c, n);
                    ll diff = x > y ? x - y : y - x;
                    q = mult(q, diff, n);
                }
                g = __gcd(q, n);
                k += m;
            }
            r <= 1;
        }
        if (g == n) {
            do {
                ys = f(ys, c, n);
                ll diff = x > ys ? x - ys : ys - x;
                g = __gcd(diff, n);
            } while (g == 1);
        }
        if (g > 1 && g < n) return g;
    }
}
vector<pll> fp(ll n) {
    stack<ll> st;
    st.push(n);
}

```

```

vector<pll> ans;
map<ll, ll> cont;
while (!st.empty()) {
    ll x = st.top();
    st.pop();
    if (x == 1) continue;
    if (esprimo(x)) cont[x]++;
    else {
        ll f = getfactor(x);
        st.push(f), st.push(x / f);
    }
}
for (auto [x, f] : cont) ans.pb({x, f});
return ans;
}

```

1.9 primitive root

```

int primitive_root(int p) {
    int n = p - 1; // n=phi(p) -> n=p-1 si p es primo
    vector<int> fp;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) {
            fp.pb(i);
            while (n % i == 0) n /= i;
        }
    if (n > 1) fp.pb(n);
    n = p - 1;
    bool ok = 0;
    for (int g = 2; g <= n; g++) {
        bool ok = 1;
        for (int x : fp)
            if (qp(g, n / x, p) == 1) {
                ok = 0;
                break;
            }
        if (ok) return g;
    }
    return 0;
}

```

2 Strings

2.1 prefix function

```

void prefix_function(string &s, int pf[]) {
    int n = s.size(); pf[0] = 0;
    for (int i = 1; i < n; i++) {
        int j = pf[i - 1];
        while (j > 0 && s[i] != s[j]) j = pf[j - 1];
        pf[i] = j + (s[i] == s[j]);
    }
}

```

2.2 hashing

```
struct Hashing{
    const int mod = 1e9+9;
    const int p = 367, q = 467;
    int ep[N],eq[N],prp[N],prq[N];

    int P(int a,int b){ return 1ll*a*b%mod;}
    int S(int a,int b){ return (a+b)%mod;}

    void build(string &s){
        int n = s.size();
        ep[0] = eq[0] = 1;
        prp[0] = prq[0] = 0;
        for (int i=1; i<=n; i++){
            prp[i] = S(P(prp[i-1], p), s[i-1]);
            prq[i] = S(P(prq[i-1], q), s[i-1]);
            ep[i] = P(ep[i-1], p);
            eq[i] = P(eq[i-1], q);
        }

        ll substr(int i,int len){ i++;
            ll hp = S(prp[i+len-1], mod-P(prp[i-1], ep[len]));
            ll hq = S(prq[i+len-1], mod-P(prq[i-1], eq[len]));
            return hp*mod + hq;
        }
    }
} H1, H2;
```

2.3 z function

```
void z_function(string &s,int zf[]){
    int n = s.size();
    for (int i=1,l=0,r=0; i<n; i++){
        if (i < r) zf[i] = min(zf[i-l], r-i);
        while (i+zf[i] < n && s[i+zf[i]] == s[zf[i]]) zf[i]++;
        if (i+zf[i] > r) l = i, r = i+zf[i];
    }
}
```

3 Graphs

3.1 tree bridge

```
vector<int> adj[MX];
int tin[MX], low[MX], timer;
vector<pii> bridges;
int component[MX];
vector<int> nods[MX];
vector<array<int, 3>> adjcomp[MX];
void dfs(int u, int p) {
    tin[u] = low[u] = ++timer;
    for (int v : adj[u])
```

```
        if (v != p) {
            if (tin[v] < low[u]) low[u] = min(low[u], tin[v]);
            else dfs(v, u), low[u] = min(low[u], low[v]);
            if (tin[u] < low[v]) bridges.pb({u, v});
        }
    }
    bool isbridge(int u, int v) {
        if (tin[u] > tin[v]) swap(u, v);
        return tin[u] < low[v];
    }
    void dfsc(int u, int comp) {
        component[u] = comp;
        nods[comp].pb(u);
        for (int v : adj[u])
            if (!isbridge(u, v) && !component[v]) dfsc(v, comp);
    }
    int tree_bridge(int n) {
        dfs(1, 1);
        int comp = 0;
        for (int u = 1; u <= n; u++)
            if (!component[u]) dfsc(u, ++comp);
        for (auto [u, v] : bridges) {
            int cu = component[u], cv = component[v];
            adjcomp[cu].pb({cv, u, v});
            adjcomp[cv].pb({cu, u, v});
        }
        return comp;
    }
}
```

3.2 2sat

```
struct SAT { // Index_1
    vector<vector<int>> adj, adjt;
    vector<bool> used;
    vector<int> comp;
    int n;
    stack<int> st;
    SAT(int n) : n(n) {
        adj.resize(2 * n + 1), adjt.resize(2 * n + 1);
        used.assign(2 * n + 1, 0), comp.assign(2 * n + 1, 0);
    }
    void add(int p, bool bp, int q, bool bq) { // add p -> q
        p += (!bp) * n;
        q += (!bq) * n;
        adj[p].pb(q);
        adjt[q].pb(p);
    }
    void addor(int p, bool bp, int q, bool bq) { // add p or q
        add(p, !bp, q, bq);
        add(q, !bq, p, bp);
    }
    void addxor(int p, bool bp, int q, bool bq) { // add p xor q
```

```
        addor(p, bp, q, bq);
        addor(p, !bp, q, !bq);
    }
    void addnxor(int p, bool bp, int q, bool bq) { // add ~(p xor q)
        addor(p, !bp, q, bq);
        addor(p, bp, q, !bq);
    }
    void dfs(int u) {
        used[u] = 1;
        for (int v : adj[u])
            if (!used[v]) dfs(v);
        st.push(u);
    }
    void _dfs(int u, int c) {
        comp[u] = c;
        for (int v : adjt[u])
            if (!comp[v]) _dfs(v, c);
    }
    bool solve(vector<bool> &ans) {
        ans.resize(n + 1);
        for (int u = 1; u <= 2 * n; u++)
            if (!used[u]) dfs(u);
        int c = 0;
        while (!st.empty()) {
            int u = st.top();
            st.pop();
            if (!comp[u]) _dfs(u, ++c);
        }
        for (int p = 1; p <= n; p++) {
            if (comp[p] == comp[p + n]) return false;
            ans[p] = comp[p] > comp[p + n];
        }
        return true;
    }
};
```

3.3 dinic

```
// This implementation is for small graphs G(V, E) (|V| <= 3000 aprox)
// where 0<|V|^2 memory will fit the memory limit.
using Cap = long long;
const int MX = 3000;
const Cap INF = 1e18;
struct Graph {
    vector<int> adj[MX];
    Cap cap[MX][MX];
    Cap flow[MX][MX];
    bool added[MX][MX];
    int level[MX];
    int nextEdge[MX];

    void clear(int n) {
        for (int i = 0; i < n; i++) {
```

```

adj[i].clear();
for (int j = 0; j < n; j++) cap[i][j] = 0,
flow[i][j] = 0, added[i][j] = false;
}

void addEdge(int u, int v, Cap w, bool dir) {
    if (!added[u][v]) {
        adj[u].push_back(v);
        adj[v].push_back(u);
        added[u][v] = added[v][u] = true;
    }
    cap[u][v] += w;
    if (!dir) cap[v][u] += w;
}

Cap dfs(int u, int t, Cap f) { // 0(E)
    if (u == t) return f;
    for (int &i = nextEdge[u]; i < adj[u].size(); i++) {
        int v = adj[u][i];
        Cap cf = cap[u][v] - flow[u][v];
        if (cf == 0 || level[v] != level[u] + 1)
            continue;
        Cap ret = dfs(v, t, min(f, cf));
        if (ret > 0) {
            flow[u][v] += ret;
            flow[v][u] -= ret;
            return ret;
        }
    }
    return 0;
}

bool bfs(int s, int t, int n) { // 0(E)
    fill(level, level + n, -1);
    queue<int> q({s});
    level[s] = 0;
    while (!q.empty()) {
        int u = q.front();
        nextEdge[u] = 0;
        q.pop();
        for (int v : adj[u]) {
            Cap cf = cap[u][v] - flow[u][v];
            if (level[v] == -1 && cf > 0) {
                level[v] = level[u] + 1;
                q.push(v);
            }
        }
    }
    return level[t] != -1;
}

Cap maxFlow(int s, int t, int n) {
    // General: 0(E * V^2), 0(E * V + V * |F|)
    // Unit Cap: 0(E * min(E^(1/2), V^(2/3)))

```

```

// Unit Network: 0(E * V^(1/2))
// In unit network, all the edges have unit capacity
// and for any vertex except s and t either the
// incoming or outgoing edge is unique.
Cap ans = 0;
while (bfs(s, t, n)) { // 0(V) iterations
    // after bfs, the graph is a DAG
    while (Cap inc = dfs(s, t, INF)) ans += inc;
}
return ans;
}
} G;

```

4 Data Structure

4.1 2d fenwick tree

```

struct BIT2D {
    vector<vector<int>> t;
    int n, m;
    BIT2D(int n, int m) : n(n), m(m) {
        t.assign(n, vector<int>(m, 0));
    }
    void update(int x, int y, int add) {
        for (int i = x; i < n; i |= i + 1)
            for (int j = y; j < m; j |= j + 1) t[i][j] += add;
    }
    int sum(int x, int y) {
        int ans = 0;
        for (int i = x; i >= 0; i = (i & (i + 1)) - 1)
            for (int j = y; j >= 0; j = (j & (j + 1)) - 1)
                ans += t[i][j];
        return ans;
    }
    int query(int lx, int rx, int ly, int ry) {
        return sum(rx, ry) - sum(lx - 1, ry) - sum(rx, ly - 1) + sum(lx - 1, ly - 1);
    }
};

```

4.2 dynamic segment tree

```

struct nod {
    ll t;
    ll lazy;
    nod *l;
    nod *r;
};

static nod nods[ND];
static int id = 0;
nod *crearnodo() {
    nods[id].t = NEUT, nods[id].lazy = NEUT_LAZY, nods[id].l
= nods[id].r = NULL;
}

```

```

return &nods[id++];
}

ll f(ll a, ll b) {
    return a + b;
}

void aply(ll val, nod *u, ll l, ll r) {
    u->t = (r - l + 1) * val;
    u->lazy = val;
}

struct segmentTree {
    ll n;
    nod *root = NULL;
    segmentTree(ll n) : n(n) {}
    void push(nod *u, ll l, ll r) {
        if (u->lazy == NEUT_LAZY || l == r) return;
        ll m = (l + r) / 2;
        if (!u->l) u->l = crearnodo();
        if (!u->r) u->r = crearnodo();
        aply(u->lazy, u->l, l, m);
        aply(u->lazy, u->r, m + 1, r);
        u->lazy = NEUT_LAZY;
    }
    ll query(ll ql, ll qr, nod *u, ll l, ll r) {
        if (qr < l || r < ql || !u) return NEUT;
        if (ql <= l && r <= qr) return u->t;
        push(u, l, r);
        ll m = (l + r) / 2;
        return f(query(ql, qr, u->l, l, m), query(ql, qr, u->r, m + 1, r));
    }
    ll query(ll ql, ll qr) {
        return query(ql, qr, root, 0, n - 1);
    }
    void update(ll ql, ll qr, ll val, nod *u, ll l, ll r) {
        if (qr < l || r < ql) return;
        if (!u) u = crearnodo();
        if (ql <= l && r <= qr) aply(val, u, l, r);
        else {
            push(u, l, r);
            ll m = (l + r) / 2;
            update(ql, qr, val, u->l, l, m);
            update(ql, qr, val, u->r, m + 1, r);
            u->t = f(u->l ? u->l->t : NEUT, u->r ? u->r->t : NEUT);
        }
    }
    void update(ll ql, ll qr, ll val) {
        update(ql, qr, val, root, 0, n - 1);
    }
};

```

4.3 pst

```

const ll MX = 2e5 + 5;
const ll Q = 2e5 + 5;

```

```

const ll MXT = 2 * MX + 20 * Q;
struct PST {
    ll root[Q + 1], t[MXT], L[MXT], R[MXT];
    ll n, nodos, lastversion = 0;
    PST(vector<ll> &a) : n(a.size()) {
        nodos = 2 * n - 1;
        root[0] = 1;
        build(a, 1, 0, n - 1);
    }
    ll f(ll a, ll b) {
        return a + b;
    }
    void build(vector<ll> &a, ll id, ll l, ll r) {
        if (l == r) t[id] = a[l];
        else {
            ll m = (l + r) / 2;
            ll idl = id + 1;
            ll idr = id + 2 * (m - l + 1);
            L[id] = idl;
            R[id] = idr;
            build(a, idl, l, m);
            build(a, idr, m + 1, r);
            t[id] = f(t[idl], t[idr]);
        }
    }
    ll query(ll ql, ll qr, ll id, ll l, ll r) {
        if (ql <= l && r <= qr) return t[id];
        ll m = (l + r) / 2;
        if (qr <= m) return query(ql, qr, L[id], l, m);
        if (m + 1 <= ql) return query(ql, qr, R[id], m + 1, r);
        return f(query(ql, qr, L[id], l, m), query(ql, qr, R[id], m + 1, r));
    }
    ll query(ll ql, ll qr, ll version = -1) {
        if (version == -1) version = lastversion;
        return query(ql, qr, root[version], 0, n - 1);
    }
    void update(ll pos, ll val, ll prev, ll &id, ll l, ll r) {
        id = ++nodos;
        if (l == r) t[id] = val;
        else {
            ll m = (l + r) / 2;
            L[id] = L[prev];
            R[id] = R[prev];
            if (pos <= m) update(pos, val, L[prev], L[id], l, m);
            else update(pos, val, R[prev], R[id], m + 1, r);
            t[id] = f(t[L[id]], t[R[id]]);
        }
    }
    void update(ll pos, ll val, ll version = -1) {
        if (version == -1) version = lastversion;
        lastversion++;
    }

```

```

        update(pos, val, root[version], root[lastversion],
0, n - 1);
    }
};

```

4.4 segment tree 2d

```

struct ST2D {
    int n, m;
    vector<vector<ll>> t;
    ll f(ll a, ll b) {
        return a + b;
    }
    ST2D(vector<vector<ll>> &a) {
        n = a.size();
        m = a[0].size();
        t.assign(2 * n, vector<ll>(2 * m, 0));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) t[i + n][j + m] = a[i][j];
        for (int i = n; i < 2 * n; i++)
            for (int j = m - 1; j > 0; j--) t[i][j] = f(t[i][j < 1], t[i][j < 1 | 1]);
        for (int i = n - 1; i > 0; i--)
            for (int j = 1; j < 2 * m; j++) t[i][j] = f(t[i < 1][j], t[i < 1 | 1][j]);
    }
    void update(int x, int y, ll val) {
        x += n, y += m, t[x][y] = val;
        for (int j = y; j > 1; j >= 1) t[x][j > 1] = f(t[x][j], t[x][j ^ 1]);
        for (int i = x > 1; i > 0; i >= 1) {
            int j = y;
            t[i][j] = f(t[i < 1][j], t[i < 1 | 1][j]);
            for (; j > 1; j >= 1) t[i][j > 1] = f(t[i][j], t[i][j ^ 1]);
        }
    }
    ll queryY(int i, int l, int r) {
        ll ans = 0;
        for (l += m, r += m + 1; l < r; l >= 1, r >= 1) {
            if (l & 1) ans = f(ans, t[i][l++]);
            if (r & 1) ans = f(t[i][--r], ans);
        }
        return ans;
    }
    ll query(int lx, int rx, int ly, int ry) {
        ll ans = 0;
        for (x1 += n, x2 += n + 1; x1 < x2; x1 >= 1, x2 >= 1) {
            if (x1 & 1) ans = f(ans, queryY(x1++, y1, y2));
            if (x2 & 1) ans = f(queryY(--x2, y1, y2), ans);
        }
        return ans;
    }

```

```

    }
};

```

4.5 merge sort tree

```

struct segmentTree {
    vector<vector<ll>> seg;
    ll n, N;
    segmentTree() {}
    segmentTree(vector<ll> &a) : n(a.size()) {
        N = 1;
        while (N < n) N <= 1;
        seg.assign(2 * N, vector<ll>());
        for (int i = 0; i < n; i++) seg[i + N] = {a[i]};
        for (int i = N - 1; i > 0; i--) {
            auto &l = seg[i < 1], &r = seg[i < 1 | 1];
            seg[i].resize((ll)l.size() + (ll)r.size());
            merge(all(l), all(r), seg[i].begin());
        }
    }
    // cantidad de elementos mayores a "a" en el intervalo [ql,qr]
    ll query1(ll ql, ll qr, ll a) {
        ll l = ql + N, r = qr + N;
        ll ans = 0;
        while (l <= r) {
            if (l & 1) {
                auto &v = seg[l++];
                ans += v.end() - upper_bound(all(v), a);
            }
            if (!(r & 1)) {
                auto &v = seg[r--];
                ans += v.end() - upper_bound(all(v), a);
            }
            l >= 1;
            r >= 1;
        }
        return ans;
    }
    // cantidad de elementos menores a "a" en el intervalo [ql,qr]
    ll query2(ll ql, ll qr, ll a) {
        ll l = ql + N, r = qr + N;
        ll ans = 0;
        while (l <= r) {
            if (l & 1) {
                auto &v = seg[l++];
                ans += lower_bound(all(v), a) - v.begin();
            }
            if (!(r & 1)) {
                auto &v = seg[r--];
                ans += lower_bound(all(v), a) - v.begin();
            }
            l >= 1;
            r >= 1;
        }
    }

```

```

    }
    return ans;
}
};

```

4.6 ordered set

```

// find_by_order(k) -> return iterator to the k-th element
// (0-indexed) - 0(log n)
// order_of_key(num) -> # of items strictly smaller than num
// - 0(log n)
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

ordered_set<int> S;

```

4.7 sparse table

```

const int LG = __lg(MX) + 1;
int st[MX][LG];
void build(vector<int> &A) { // O(|f| n log n)
    int n = A.size();
    for (int i = 0; i < n; i++) st[i][0] = A[i];
    for (int k = 1; k < LG; k++) {
        for (int i = 0; i + (1 << k) <= n; i++) {
            st[i][k] = min(st[i][k - 1], st[i + (1 << (k - 1))][k - 1]);
        }
    }
}
int query(int l, int r) { // O(|f|)
    int lg = __lg(r - l + 1);
    return min(st[l][lg], st[r - (1 << lg) + 1][lg]);
}

```

4.8 pst lazy

```

const int MX = 2e5 + 5;
const int Q = 2e5 + 5;
const int LOG = 20;
const int ND = 2 * MX + 6 * Q * LOG;
const ll NEUT = 0;
const ll NEUT_LAZY = -2 * INF;
struct nod {
    ll t, lazy;
    nod *l, *r;
};
nod nods[ND];
int id = 0;
nod *crearnodo(nod *u = NULL) {

```

```

    if (u) nods[id].t = u->t, nods[id].lazy = u->lazy,
    nods[id].l = u->l, nods[id].r = u->r;
    else nods[id].t = NEUT, nods[id].lazy = NEUT_LAZY,
    nods[id].l = nods[id].r = NULL;
    return &nods[id++];
}
ll f(ll a, ll b) {
    return a + b;
}
void apply(ll val, nod *u, ll l, ll r) {
    u->t += (r - l + 1) * val;
    if (u->lazy == NEUT_LAZY) u->lazy = val;
    else u->lazy += val;
}
struct PST {
    nod *root[MX + 5];
    ll n;
    ll lastversion = -1;
    PST(vector<ll> &a) : n(a.size()) {
        root[++lastversion] = crearnodo();
        build(a, root[0], 0, n - 1);
    }
    void build(vector<ll> &a, nod *u, ll l, ll r) {
        if (!u) u = crearnodo();
        if (l == r) u->t = a[l];
        else {
            ll m = (l + r) >> 1;
            build(a, u->l, l, m);
            build(a, u->r, m + 1, r);
            u->t = f(u->l->t, u->r->t);
        }
    }
    void push(nod *u, ll l, ll r) {
        if (u->lazy == NEUT_LAZY || l == r) return;
        ll m = (l + r) >> 1;
        u->l = crearnodo(u->l), u->r = crearnodo(u->r);
        apply(u->lazy, u->l, l, m);
        apply(u->lazy, u->r, m + 1, r);
        u->lazy = NEUT_LAZY;
    }
    nod *update(ll ql, ll qr, ll val, nod *prev, ll l, ll r) {
        if (qr < l || r < ql) return prev;
        nod *u = crearnodo(prev);
        if (ql <= l && r <= qr) apply(val, u, l, r);
        else {
            push(u, l, r);
            ll m = (l + r) >> 1;
            u->l = update(ql, qr, val, u->l, l, m);
            u->r = update(ql, qr, val, u->r, m + 1, r);
            u->t = f(u->l->t, u->r->t);
        }
        return u;
    }
    void update(ll ql, ll qr, ll val, ll version = -1) {

```

```

        if (version == -1) version = lastversion;
        root[++lastversion] = update(ql, qr, val,
        root[version], 0, n - 1);
    }
    ll query(ll ql, ll qr, nod *u, ll l, ll r) {
        if (!u || qr < l || r < ql) return NEUT;
        if (ql <= l && r <= qr) return u->t;
        push(u, l, r);
        ll m = (l + r) >> 1;
        return f(query(ql, qr, u->l, l, m), query(ql, qr, u->r, m + 1, r));
    }
    ll query(ll ql, ll qr, ll version = -1) {
        if (version == -1) version = lastversion;
        return query(ql, qr, root[version], 0, n - 1);
    }
};

```

4.9 fenwick tree

```

struct FenwickTree { // Index 1
    int n, ft[MX + 1];
    void update(int l, int v) {
        for (; l <= n; l += l & -l) ft[l] += v;
    }
    int query(int l) {
        int ans = 0;
        for (; l > 0; l -= l & -l) ans += ft[l];
        return ans;
    }
    int query(int l, int r) { return query(r) - query(l - 1); }
};

```

4.10 centroid decomposition

```

struct Centroid_Decomposition{
    vector<int> adj[N];
    int sz[N], vis[N], aux;
    int pa[N];

    int find(int u, int p, int Sz){
        int mx = 0, c = 0; sz[u] = 1;
        for (auto v : adj[u]){
            if (v == p || vis[v]) continue;
            c ^= find(v, u, Sz);
            sz[u] += sz[v];
            mx = max(mx, sz[v]);
        }
        if (max(mx, Sz - sz[u]) <= Sz/2)
            aux = p, c = u;
        return c;
    }

    void build(int u, int p, int Sz){

```

```

    aux = -1, u = find(u, -1, Sz);
    if (aux != -1) sz[aux] = Sz-sz[u];
    vis[u] = 1, sz[u] = Sz, pa[u] = p;
    for (auto v : adj[u])
        if (!vis[v]) build(v, u, sz[v]);
}
} cd; // by NekoRolly

```

4.11 multi ordered set

```

// find_by_order(k) -> return iterator to the k-th element
// (0-indexed) - O(log n)
// order_of_key(num) -> # of items strictly smaller than num
// - O(log n)

```

```

// using less_equal cause that lower_bound works as
// upper_bound and viceversa
// also .find() does not work and you can only erase by
// iterator now
// using for example erase(upper_bound(x))
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

```

```

template <typename T>
using ordered_set =
    tree<T, null_type, less_equal<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

```

```
ordered_set<int> s;
```

4.12 hld

```

struct Graph {
    vector<vector<ll>> adj;
    vector<ll> padre, prof, heavy, head, pos, base, L, R;
    segmentTree st;
    ll curpos, n;
    ll dfs(ll u, ll p) {
        padre[u] = p;
        ll szu = 1;
        ll mxsz = 0;
        heavy[u] = 0;
        for (ll v : adj[u])
            if (v != p) {
                prof[v] = prof[u] + 1;
                ll szv = dfs(v, u);
                szu += szv;
                if (mxsz < szv) {
                    mxsz = szv;
                    heavy[u] = v;
                }
            }
        return szu;
    }
    void decompose(ll u, ll h, vector<ll> &a) {

```

```

        head[u] = h;
        L[u] = curpos;
        pos[u] = curpos++;
        base[pos[u]] = a[u - 1];
        if (heavy[u] != 0) decompose(heavy[u], h, a);
        for (ll v : adj[u])
            if (v != padre[u] && v != heavy[u]) decompose(v,
v, a);
        R[u] = curpos - 1;
    }
    void pupdate(ll u, ll val) {
        st.pupdate(pos[u], val);
    }
    void update(ll u, ll v, ll val) {
        while (head[u] != head[v]) {
            if (prof[head[u]] > prof[head[v]]) swap(u, v);
            st.update(pos[head[v]], pos[v], val);
            v = padre[head[v]];
        }
        if (prof[u] > prof[v]) swap(u, v);
        st.update(pos[u], pos[v], val);
    }
    ll query(ll u, ll v) {
        ll ans = INF;
        while (head[u] != head[v]) {
            if (prof[head[u]] > prof[head[v]]) swap(u, v);
            ans = f(ans, st.query(pos[head[v]], pos[v]));
            v = padre[head[v]];
        }
        if (prof[u] > prof[v]) swap(u, v);
        ans = f(ans, st.query(pos[u], pos[v]));
        return ans;
    }
    ll subtreequery(ll u) {
        return st.query(L[u], R[u]);
    }
    void subtreeupdate(ll u, ll val) {
        st.update(L[u], R[u], val);
    }
    Graph(vector<ll> &a, vector<pll> &aristas, ll root =
1) : n(a.size()) {
        adj.assign(n + 1, vector<ll>()), padre.assign(n + 1,
0), prof.assign(n + 1, 0);
        heavy.assign(n + 1, 0), head.assign(n + 1, 0),
pos.assign(n + 1, 0);
        L.assign(n + 1, 0), R.assign(n + 1, 0),
base.resize(n);
        curpos = 0;
        for (auto [u, v] : aristas) {
            adj[u].pb(v);
            adj[v].pb(u);
        }
        prof[root] = 0;
        dfs(root, root);
        decompose(root, root, a);

```

```

        st = segmentTree(base);
    }
};

```

4.13 segment tree

```

struct IterativeSegmentTree { // Index 0
    int n;
    ll ST[2 * MX];
    const ll NEUT = 0;
    void build(vector<ll> &a) {
        n = a.size();
        for (int i = n; i < (n << 1); i++) ST[i] = a[i - n];
        for (int i = n - 1; i > 0; i--) ST[i] = f(ST[i <<
1], ST[i << 1 | 1]);
    }
    ll f(ll x, ll y) { return max(x, y); }
    void update(int i, ll val) {
        for (ST[i += n] = val; i >= 1; i >>= 1) ST[i] = f(ST[i <<
1], ST[i << 1 | 1]);
    }
    ll query(int l, int r) { // [l, r)
        ll ansL = NEUT, ansR = NEUT;
        for (l += n, r += n; l < r; l >= 1, r >= 1) {
            if (l & 1) ansL = f(ansL, ST[l++]);
            if (r & 1) ansR = f(ST[--r], ansR);
        }
        return f(ansL, ansR);
    }
};

```

4.14 hld edge

```

struct Graph {
    vector<vector<pll>> adj;
    vector<ll> padre, wpadre, prof, heavy, head, pos, base,
L, R;
    segmentTree st;
    ll curpos;
    ll n;
    ll dfs(ll u, ll p, ll wp) {
        padre[u] = p;
        wpadre[u] = wp;
        ll szu = 1;
        ll mxsz = 0;
        heavy[u] = 0;
        for (auto [v, w] : adj[u])
            if (v != p) {
                prof[v] = prof[u] + 1;
                ll szv = dfs(v, u, w);
                szu += szv;
                if (mxsz < szv) mxsz = szv, heavy[u] = v;
            }
        return szu;
    }
};

```

```

void decompose(ll u, ll h) {
    head[u] = h;
    L[u] = curpos;
    pos[u] = curpos++;
    base[pos[u]] = wpadre[u];
    if (heavy[u] != 0) decompose(heavy[u], h);
    for (auto [v, w] : adj[u])
        if (v != padre[u] && v != heavy[u]) decompose(v,
v);
    R[u] = curpos - 1;
}

void pupdate(ll u, ll v, ll val) {
    if (u == padre[v]) swap(u, v);
    st.pupdate(pos[u], val);
}

void update(ll u, ll v, ll val) {
    while (head[u] != head[v]) {
        if (prof[head[u]] > prof[head[v]]) swap(u, v);
        st.update(pos[head[v]], pos[v], val);
        v = padre[head[v]];
    }
    if (prof[u] > prof[v]) swap(u, v);
    if (u != v) st.update(pos[u] + 1, pos[v], val);
}

ll query(ll u, ll v) {
    ll ans = INF;
    while (head[u] != head[v]) {
        if (prof[head[u]] > prof[head[v]]) swap(u, v);
        ans = f(ans, st.query(pos[head[v]], pos[v]));
        v = padre[head[v]];
    }
    if (prof[u] > prof[v]) swap(u, v);
    if (u != v) ans = f(ans, st.query(pos[u] + 1,
pos[v]));
    return ans;
}

ll subtreequery(ll u) {
    if (L[u] == R[u]) return 0;
    return st.query(L[u] + 1, R[u]);
}

void subtreeupdate(ll u, ll val) {
    if (L[u] == R[u]) return;
    st.update(L[u] + 1, R[u], val);
}

Graph(vector<pair<pll, ll>> &aristas, ll root = 1) :
n(aristas.size() + 1) {
    adj.assign(n + 1, vector<pll>());
    padre.assign(n + 1, 0), wpadre.assign(n + 1, 0),
prof.assign(n + 1, 0);
    heavy.assign(n + 1, 0), head.assign(n + 1, 0),
pos.assign(n + 1, 0);
    L.assign(n + 1, 0), R.assign(n + 1, 0),
base.resize(n);
    curpos = 0;
    for (auto [edge, w] : aristas) {

```

```

        auto [u, v] = edge;
        adj[u].pb({v, w});
        adj[v].pb({u, w});
    }
    prof[root] = 0;
    dfs(root, root, 0LL);
    decompose(root, root);
    st = segmentTree(base);
}
};

```